

Lab Assignment 3: Cryptology Lab - Introduction to Python

Part A - Python Basics and Debugging

1. Debug the following programs. Identify the type of error in each case.

```
print("Welcome to Cryptology Lab")  
  
message = "Python Programming"  
  
print(mesage)  
  
num = input("Enter a number: ")  
  
result = num + 10  
  
print(result)
```

```
print("Welcome to Cryptology Lab") #Syntax error  
message = "Python Programming"  
print(message) #Name error  
num = int(input("Enter a number: ")) #Type error  
result = num + 10  
print(result)  
  
Welcome to Cryptology Lab  
Python Programming  
Enter a number: 10  
20
```

2. Write Python statements to display the following exactly:

Welcome to Cryptology Lab

It's interesting to learn Python.

Students, "Welcome to the Cryptology Lab".

```
print("Welcome to Cryptology Lab\nIt's interesting to learn Python.\nStudents, \"Welcome to the Cryptology Lab\".")  
  
Welcome to Cryptology Lab  
It's interesting to learn Python.  
Students, "Welcome to the Cryptology Lab".
```

3. Read two numbers from the user and perform:

- Addition
 - Subtraction
 - Multiplication
 - Division
 - Integer division
 - Remainder
 - Exponentiation
- Display each result with an appropriate message.

```

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("Addition:", num1 + num2)
print("Subtraction:", num1 - num2)
print("Multiplication:", num1 * num2)
print("Division:", num1 / num2)
print("Integer Division:", num1 // num2)
print("Remainder:", num1 % num2)
print("Exponentiation:", num1 ** num2)

```

```

Enter first number: 10
Enter second number: 5
Addition: 15.0
Subtraction: 5.0
Multiplication: 50.0
Division: 2.0
Integer Division: 2.0
Remainder: 0.0
Exponentiation: 100000.0

```

4. Read a floating-point number and display:

its data type;

its absolute value;

the value rounded to two decimal places.

```

num = float(input("Enter a floating-point number: "))

print("Data type:", type(num))
print("Absolute value:", abs(num))
print("Rounded value:", round(num, 2))

```

```

Enter a floating-point number: -14.004
Data type: <class 'float'>
Absolute value: 14.004
Rounded value: -14.0

```

Part B - Data Types and Collections (~25 min)

5. Create variables containing examples of:

int, float, complex, str, list, tuple, set, and dictionary.

Use Python to display the value and data type of each variable.

```

a = 10
b = 10.4
c = 2 + 3j
d = "Cryptology"
e = [1, 2, 3]
f = (1, 2, 3)
g = {1, 2, 3}
h = {"name": "Praneetha", "age": 22}

print(a, type(a))
print(b, type(b))
print(c, type(c))
print(d, type(d))
print(e, type(e))
print(f, type(f))
print(g, type(g))
print(h, type(h))

10 <class 'int'>
10.4 <class 'float'>
(2+3j) <class 'complex'>
Cryptology <class 'str'>
[1, 2, 3] <class 'list'>
(1, 2, 3) <class 'tuple'>
{1, 2, 3} <class 'set'>
{'name': 'Praneetha', 'age': 22} <class 'dict'>

```

6. Demonstrate experimentally that:
Strings are immutable, whereas lists are mutable.

Include the Python statements you used and explain the observed result.

```
# String
message = "HELLO"
print("Original string:", message)

# modify a character
message[0] = "Y" # This gives TypeError

# List
letters = ["H", "E", "L", "L", "O"]
print("Original list:", letters)

letters[0] = "Y"
print("Modified list:", letters)
```

Original string: HELLO

[Fix Code](#)

TypeError

Traceback (most recent call last)

Cell In[10], line 6

```
3 print("Original string:", message)
5 # modify a character
----> 6 message[0] = "Y" # This gives TypeError
8 # List
9 letters = ["H", "E", "L", "L", "O"]
```

TypeError: 'str' object does not support item assignment

```
# String
message = "HELLO"
print("Original string:", message)

# modify a character
#message[0] = "Y" # This gives TypeError

# List
letters = ["H", "E", "L", "L", "O"]
print("Original list:", letters)

letters[0] = "Y"
print("Modified list:", letters)
```

Original string: HELLO
Original list: ['H', 'E', 'L', 'L', 'O']
Modified list: ['Y', 'E', 'L', 'L', 'O']

Strings are immutable, so individual characters cannot be changed. Lists are mutable, so their elements can be modified.

7. Create a list containing five prime numbers. Perform the following:

- display the first three elements;
- display the last element;
- append another prime number;
- extend the list with two more prime numbers;
- display the total number of elements.

```
primes = [2, 3, 5, 7, 11]

print("First three elements:", primes[:3])
print("Last element:", primes[-1])

primes.append(13)
print("After append:", primes)

primes.extend([17, 19])
print("After extend:", primes)

print("Total number of elements:", len(primes))
```

First three elements: [2, 3, 5]
Last element: 11
After append: [2, 3, 5, 7, 11, 13]
After extend: [2, 3, 5, 7, 11, 13, 17, 19]
Total number of elements: 8

Part C - String Operations

For Questions 8–12, use:

message = "CRYPTOLOGY USING PYTHON"

8. Display:
- length of the string;
 - first five characters;
 - last six characters;

characters from positions 3 to 8;

string in reverse order.

```
message = "CRYPTOLOGY USING PYTHON"

print("Length:", len(message))
print("First five characters:", message[:5])
print("Last six characters:", message[-6:])
print("Characters from positions 3 to 8:", message[3:9])
print("Reverse:", message[::-1])
```

```
Length: 23
First five characters: CRYPT
Last six characters: PYTHON
Characters from positions 3 to 8: PTOLOG
Reverse: NOHTYP GNISU YGOLOTPYRC
```

9. Perform the following:

convert to lowercase;

convert back to uppercase;

check whether "PYTHON" exists in the string;

replace "PYTHON" with "PROGRAMMING".

```
message = "CRYPTOLOGY USING PYTHON"

print("Lowercase:", message.lower())
print("Uppercase:", message.upper())

print("PYTHON exists:", "PYTHON" in message)

print("After replacement:",
      message.replace("PYTHON", "PROGRAMMING"))

Lowercase: cryptography using python
Uppercase: CRYPTOLOGY USING PYTHON
PYTHON exists: True
After replacement: CRYPTOLOGY USING PROGRAMMING
```

10. Remove the spaces from the string and display the resulting string.

```
message = "CRYPTOLOGY USING PYTHON"
result = message.replace(" ", "")
print("String without spaces:", result)

String without spaces: CRYPTOLOGYUSINGPYTHON
```

11. Using a loop, display every character and its corresponding Unicode/ASCII value using `ord()`.

Example:

C -> 67

R -> 82

Y -> 89

...

```

message = "CRYPTOLOGY USING PYTHON"

for ch in message:
    print(ch, "->", ord(ch))

C -> 67
R -> 82
Y -> 89
P -> 80
T -> 84
O -> 79
L -> 76
O -> 79
G -> 71
Y -> 89
-> 32
U -> 85
S -> 83
I -> 73
N -> 78
G -> 71
-> 32
P -> 80
Y -> 89
T -> 84
H -> 72
O -> 79
N -> 78

```

12. Use chr() to convert the following values into characters:

65, 67, 73, 80, 72, 69, 82

Combine the resulting characters and identify the word.

```

values = [65, 67, 73, 80, 72, 69, 82]

word = ""

for value in values:
    word += chr(value)

print(word[0] + " " + word[1:])

A CIPHER

```

Part D - Text Processing for Cryptology

13. Plaintext preprocessing

Read a message from the user and:

remove leading/trailing spaces;

convert it to uppercase;

remove spaces between words.

Example:

Input: meet me tomorrow

Output: MEETMETOMORROW

```

message = input("Enter a message: ")

message = message.strip()
message = message.upper()
message = message.replace(" ", "")

print("Processed message:", message)

Enter a message: meet me tomorrow
Processed message: MEETMETOMORROW

```

14. Character grouping

Read a string and divide it into blocks of five characters.

Example:

Input: CRYPTOLOGYLAB

Output:

CRYPT

LOGY

LAB

```
message = input("Enter a string: ")

for i in range(0, len(message), 5):
    print(message[i:i+5])
```

```
Enter a string:  CRYPTOLOGYLAB
CRYPT
LOGY
LAB
```

15. Padding

Modify Question 14 so that an incomplete final block is padded with X.

Input: CRYPTOLOGYLAB

Output:

CRYPT

LOGY

LABXX

```
message = input("Enter a string: ")

while len(message) % 5 != 0:
    message += "X"

for i in range(0, len(message), 5):
    print(message[i:i+5])
```

```
Enter a string:  CRYPTOLOGYLAB
CRYPT
LOGY
LABXX
```

16. Character frequency

For the following text:

text = "CRYPTOLOGYISINTERESTING"

write a program to calculate the frequency of each character.

Then identify the most frequently occurring character.

```

text = "CRYPTOLOGYISINTERESTING"

frequency = {}

for ch in text:
    if ch in frequency:
        frequency[ch] += 1
    else:
        frequency[ch] = 1

print("Frequency:", frequency)

highest = max(frequency.values())

print("Most frequent:", end=" ")
for ch, count in frequency.items():
    if count == highest:
        print(ch, end=" ")

Frequency: {'C': 1, 'R': 2, 'Y': 2, 'P': 1, 'T': 3, 'O': 2, 'L': 1, 'G': 2, 'I': 3, 'S': 2, 'N': 2, 'E': 2}
Most frequent: T I

```

17. Percentage frequency

Extend Question 16 to calculate:

Percentage Frequency=

Total Number of Characters/ Frequency of Character ×100

Display the percentage rounded to two decimal places.

```

text = "CRYPTOLOGYISINTERESTING"

frequency = {}

for ch in text:
    if ch in frequency:
        frequency[ch] += 1
    else:
        frequency[ch] = 1

total = len(text)

for ch, count in frequency.items():
    percentage = (total / count) * 100
    print(ch, ":", round(percentage, 2), "%")

C : 2300.0 %
R : 1150.0 %
Y : 1150.0 %
P : 2300.0 %
T : 766.67 %
O : 1150.0 %
L : 2300.0 %
G : 1150.0 %
I : 766.67 %
S : 1150.0 %
N : 1150.0 %
E : 1150.0 %

```

Part E - Mathematical Foundations for Cryptology

18. Modular arithmetic

Use Python to evaluate:

$29 \bmod 26$

$55 \bmod 26$

$78 \bmod 26$

$-3 \bmod 26$

$-29 \bmod 26$

Then write a program that accepts an integer and calculates the integer modulo 26.

```
print("29 mod 26 =", 29 % 26)
print("55 mod 26 =", 55 % 26)
print("78 mod 26 =", 78 % 26)
print("-3 mod 26 =", -3 % 26)
print("-29 mod 26 =", -29 % 26)

num = int(input("Enter an integer: "))

print("Number modulo 26:", num % 26)
```

```
29 mod 26 = 3
55 mod 26 = 3
78 mod 26 = 0
-3 mod 26 = 23
-29 mod 26 = 23
Enter an integer: 144
Number modulo 26: 14
```

19. Alphabet representation

Using

A=0, B=1, ..., Z=25

write a program that accepts an uppercase character and displays its numerical representation.

Example:

Enter character: M

M -> 12

Also perform the reverse conversion:

Enter number: 12

12 -> M

```

ch = input("Enter character: ").upper()

value = ord(ch) - ord('A')

print(ch, "->", value)

num = int(input("Enter number: "))

if 0 <= num <= 25:
    ch = chr(num + ord('A'))
    print(num, "->", ch)
else:
    print("Enter a number between 0 and 25.")

Enter character: M
M -> 12
Enter number: 12
12 -> M

```

20. Prime numbers

Write a program to:

determine whether a given number is prime;

display all prime numbers within a range specified by the user.

```

num = int(input("Enter a number: "))

if num < 2:
    print("Not a prime number")
else:
    prime = True

    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            prime = False
            break

    if prime:
        print("Prime number")
    else:
        print("Not a prime number")

start = int(input("Enter starting number: "))
end = int(input("Enter ending number: "))

print("Prime numbers in the range", start, "to", end, "are: ")

for num in range(start, end + 1):
    if num >= 2:
        prime = True

        for i in range(2, int(num ** 0.5) + 1):
            if num % i == 0:
                prime = False
                break

        if prime:
            print(num, end=" ")

```

```

Enter a number: 13
Prime number
Enter starting number: 1
Enter ending number: 44
Prime numbers in the range 1 to 44 are:
2 3 5 7 11 13 17 19 23 29 31 37 41 43

```

21. GCD and coprimes

Read two integers from the user. Find their GCD and determine whether they are coprime.

Test your program with:

15 and 26

18 and 24

17 and 26

```
import math

num1 = int(input("Enter first integer: "))
num2 = int(input("Enter second integer: "))

gcd = math.gcd(num1, num2)

print("GCD:", gcd)

if gcd == 1:
    print("The numbers are coprime.")
else:
    print("The numbers are not coprime.")
```

```
Enter first integer: 15
Enter second integer: 26
GCD: 1
The numbers are coprime.
```

```
import math

num1 = int(input("Enter first integer: "))
num2 = int(input("Enter second integer: "))

gcd = math.gcd(num1, num2)

print("GCD:", gcd)

if gcd == 1:
    print("The numbers are coprime.")
else:
    print("The numbers are not coprime.")
```

```
Enter first integer: 18
Enter second integer: 24
GCD: 6
The numbers are not coprime.
```

```
import math

num1 = int(input("Enter first integer: "))
num2 = int(input("Enter second integer: "))

gcd = math.gcd(num1, num2)

print("GCD:", gcd)

if gcd == 1:
    print("The numbers are coprime.")
else:
    print("The numbers are not coprime.")
```

```
Enter first integer: 17
Enter second integer: 26
GCD: 1
The numbers are coprime.
```

22. Write a program to get the multiplicative inverse of an integer b.

```
b = int(input("Enter integer: "))

for x in range(1, 26):
    if (b * x) % 26 == 1:
        print("Multiplicative inverse =", x)
        break
else:
    print("Inverse does not exist")
```

```
Enter integer: 19
Multiplicative inverse = 11
```

Part F - Challenge / If Time Permits (~15 min)

23. WAP in python to implement client and server communication.

```
import socket
import threading

def server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(("127.0.0.1", 5000))
    server_socket.listen(1)

    print("Server is waiting for connection...")

    connection, address = server_socket.accept()
    print("Connected to:", address)

    message = connection.recv(1024).decode()
    print("Client:", message)

    connection.send("Hello from Server!".encode())

    connection.close()
    server_socket.close()
```

```
server_thread = threading.Thread(target=server)
server_thread.start()
```

```
Server is waiting for connection...
```

```
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client_socket.connect(("127.0.0.1", 5000))

message = input("Enter message for server: ")

client_socket.send(message.encode())

reply = client_socket.recv(1024).decode()

print("Server:", reply)

client_socket.close()
```

```
Connected to: ('127.0.0.1', 54912)
Enter message for server: Hello Server
Client: Hello Server
Server: Hello from Server!
```